Computational Physics Lectures

Week 01: Sampling and Markov Chains

Bjørk Hammer
Department of Physics and Astronomy
Aarhus University
Denmark

Contents

1 Sampling and Markov Chains **3**

1.1 Introduction . . . 3

1.1.1 Law of large numbers . . . 4

1.2 Sampling . . . 4

1.2.1 Uniformly and normally distributed values . . . 4

1.2.2 Uniform sampling in a two-dimensional region . . . 5

1.2.3 Sampling from a general distribution . . . 7

1.2.4 Rejection sampling . . . 7

1.3 Markov Chains . . . 9

1.3.1 From independent draws to dependent states . . . 9

1.3.2 Discrete and continuous state spaces . . . 10

1.3.3 The Markov property and transition probabilities . . . 10

1.3.4 Stationary distributions and convergence . . . 10

1.3.5 Detailed balance . . . 11

1.3.6 Example: a two-state chain . . . 12

© Bjørk Hammer

August 23, 2026

Week 1

Sampling and Markov Chains

1.1 Introduction

Many experiments in physics have stochastic outcomes. Repeating such an experiment under nominally identical conditions generally produces different results. Each repetition produces an *observation*, or *realization*, of a random variable. A collection of such observed values is called a *sample*, and the number of observations in the collection is called the *sample size*. The process of generating or measuring observations is called *sampling*.

For simplicity, first consider a continuous scalar observable X. Its probability density $p(x)$ satisfies

$$1 = \int_{x_{\min}}^{x_{\max}} p(x)\,\mathrm{d}x, \tag{1.1}$$

where the bounds may be infinite. The density itself is not a probability. Rather, for a sufficiently small interval of width Δx centred on x,

$$\Pr\left(x - \frac{\Delta x}{2} \le X < x + \frac{\Delta x}{2}\right) \simeq p(x)\,\Delta x. \tag{1.2}$$

More generally,

$$\Pr(a \le X < b) = \int_a^b p(x)\,\mathrm{d}x. \tag{1.3}$$

We use an upper-case letter such as X for a random variable and a lower-case letter such as x for one possible value or realization.

Two useful summaries of a distribution are its expectation value

$$\mu = \mathbb{E}[X] = \int x\,p(x)\,\mathrm{d}x \tag{1.4}$$

and its variance

$$\sigma^2 = \mathbb{E}\big[(X-\mu)^2\big] = \int (x-\mu)^2 p(x)\,\mathrm{d}x. \tag{1.5}$$

The standard deviation is σ.

1.1.1 Law of large numbers

Let

$$X_1, X_2, \ldots, X_N$$

be independent and identically distributed (i.i.d.) random variables, each having the same distribution as X. Assume that the expectation value

$$\mu = \mathbb{E}[X]$$

is finite.

One realization of these random variables produces the sample of size N

$$x_1, x_2, \ldots, x_N.$$

The corresponding sample mean is

$$\bar{x}_N = \frac{1}{N}\sum_{i=1}^{N} x_i.$$

Since the observed values are realizations of random variables, the sample mean is itself a random variable:

$$\overline{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i.$$

The law of large numbers states that $\overline{X}_N$ approaches μ as N becomes large. In its weak form,

$$\Pr\left(\left|\overline{X}_N - \mu\right| > \varepsilon\right) \longrightarrow 0 \qquad \text{as } N \to \infty$$

for every $\varepsilon > 0$.

If $X_1, \ldots, X_N$ are independent and each has variance σ^2, then

$$\mathrm{Var}(\overline{X}_N) = \frac{\sigma^2}{N}, \qquad \mathrm{SE}(\overline{X}_N) = \frac{\sigma}{\sqrt{N}}.$$

Thus, increasing the sample size by a factor of four reduces the standard error of the mean by a factor of two. This $N^{-1/2}$ scaling is fundamental in Monte Carlo calculations.

The law of large numbers does not imply that a finite sample reproduces the underlying distribution exactly, nor that each additional observation moves the sample mean closer to μ. Rather, it is an asymptotic statement: as N increases, the sample mean becomes increasingly likely to lie close to the expectation value.

1.2 Sampling

1.2.1 Uniformly and normally distributed values

When simulating an experiment, we often need to draw values from a known distribution. Uniform and normal distributions are directly available in NumPy. A recommended interface is to create a random-number generator explicitly:

```
import numpy as np

rng = np.random.default_rng(seed=12345)

n = 10_000
x_uniform = rng.uniform(x_min, x_max, size=n)
x_normal = rng.normal(loc=x_center, scale=sigma, size=n)
```

The call `np.random.default_rng` creates a random-number generator. Supplying a fixed seed makes the sequence reproducible, which is useful for testing and teaching. For independent production runs, the seed should be changed or omitted.

The first sampling call returns n values uniformly distributed on the half-open interval $[x_{\min}, x_{\max})$. The second returns n values from the normal distribution

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x - x_{\text{center}})^2}{2\sigma^2}\right]. \tag{1.6}$$

Here, `loc` is the mean and `scale` is the standard deviation, not the variance.

Random numbers generated by a computer are more precisely called *pseudorandom*: they are produced deterministically from an internal state but are designed to reproduce the statistical properties required for simulation.

1.2.2 Uniform sampling in a two-dimensional region

Consider the region

$$x_{\min} \le x \le x_{\max}, \qquad 0 \le y \le f(x), \tag{1.7}$$

where $f(x) > 0$ throughout the interval. Our goal is to distribute points (x, y) uniformly by area over this region.

A vertical strip of width Δx near x has area approximately $\Delta A = f(x)\Delta x$. The probability of selecting an x-value in this strip must therefore be ΔA relative to the total area, A, under the curve:

$$p_X(x)\Delta x = \frac{\Delta A}{A} = \frac{f(x)\Delta x}{A}, \qquad A = \int_{x_{\min}}^{x_{\max}} f(x')\,\mathrm{d}x'. \tag{1.8}$$

Hence

$$\boxed{p_X(x) = \frac{f(x)}{A}.} \tag{1.9}$$

Values of x for which the available vertical interval is larger must be selected more often.

Once x has been selected from $p_X(x)$, we may choose y uniformly from $[0, f(x)]$. Its conditional density is

$$p_{Y|X}(y \mid x) = \frac{1}{f(x)}, \tag{1.10}$$

as it is uniform in y and integrates to 1 for every x:

$$\int_0^{f(x)} p_{Y|X}(y \mid x)\,\mathrm{d}y = \frac{1}{f(x)} \int_0^{f(x)} \mathrm{d}y = 1. \tag{1.11}$$

The joint density is therefore

$$p_{X,Y}(x, y) = p_X(x)p_{Y|X}(y \mid x) = \frac{1}{A}, \tag{1.12}$$

which is constant throughout the allowed region. The points are consequently uniform by area.

Drawing x from the required distribution

The cumulative distribution function (CDF) associated with $p_X(x)$ is

$$F(x) = \Pr(X \le x) = \int_{x_{\min}}^{x} p_X(x')\,\mathrm{d}x'. \tag{1.13}$$

It increases from $F(x_{\min}) = 0$ to $F(x_{\max}) = 1$. If F is continuous and strictly increasing, then $U = F(X)$ is uniformly distributed on $[0, 1]$. Reversing this construction gives the inverse-transform method:

1. draw $u \sim U(0, 1)$;
2. solve $u = F(x)$ for x;
3. equivalently, set $x = F^{-1}(u)$.

Example: a linear boundary

Let

$$f(x) = b - ax, \tag{1.14}$$

with $a \ge 0$ and $b - a > 0$, so that $f(x)$ remains positive for $0 \le x \le 1$. The area is

$$A = \int_0^1 (b - ax)\,\mathrm{d}x = b - \frac{a}{2}, \tag{1.15}$$

and therefore

$$p_X(x) = \frac{b - ax}{b - a/2}. \tag{1.16}$$

The CDF is

$$F(x) = \frac{bx - ax^2/2}{b - a/2}. \tag{1.17}$$

For $a > 0$, solving $u = F(x)$ and selecting the root in $[0, 1]$ gives

$$\boxed{x = \frac{b - \sqrt{b^2 - 2a(b - a/2)u}}{a}}. \tag{1.18}$$

For $a = 0$, x is uniform and we simply take $x = u$. Finally, draw an independent $v \sim U(0, 1)$ and set

$$\boxed{y = vf(x) = v(b - ax)}. \tag{1.19}$$

The resulting pairs (x, y) are uniformly distributed over the region below $f(x)$.

1.2.3 Sampling from a general distribution

For a normalized one-dimensional density $p(x)$, inverse-transform sampling consists of

1. calculating the CDF
$$F(x) = \int_{x_{\min}}^{x} p(x')\,\mathrm{d}x'; \tag{1.20}$$
2. drawing $u \sim U(0,1)$;
3. returning $x = F^{-1}(u)$.

The upper integration limit must be x, rather than $x_{\max}$; otherwise the integral is always equal to one and contains no information about the value to be generated.

The method is convenient when F^{-1} is known analytically or can be evaluated efficiently. In more complicated cases, one may invert a numerically tabulated CDF or use alternatives such as rejection sampling or Markov-chain Monte Carlo.

Example: the exponential distribution

The exponential density is
$$p(x) = \frac{1}{\lambda}\exp(-x/\lambda), \qquad x \geq 0, \tag{1.21}$$
where $\lambda > 0$ is both the mean and the standard deviation. It describes, for example, waiting times in a Poisson process with a constant event rate.

Its CDF is
$$F(x) = \int_0^x \frac{1}{\lambda}\exp(-x'/\lambda)\,\mathrm{d}x' \tag{1.22}$$
$$= 1 - \exp(-x/\lambda). \tag{1.23}$$

Setting $u = F(x)$ and solving for x gives
$$\boxed{x = -\lambda \ln(1-u)}. \tag{1.24}$$

Since $1 - U$ has the same uniform distribution as U, one often writes $x = -\lambda \ln u$. In an implementation, care must be taken not to evaluate $\ln 0$.

1.2.4 Rejection sampling

Inverse-transform sampling is convenient when the cumulative distribution function can be evaluated and inverted efficiently. For many important distributions this is impractical. Rejection sampling provides a more general alternative.

Suppose that we wish to generate values from a normalized target density $p(x)$. We choose a proposal density $g(x)$ from which values can be drawn easily and determine a constant $M \geq 1$ such that
$$p(x) \leq Mg(x)$$

for all x in the domain. Geometrically, the curve $Mg(x)$ forms an envelope that lies everywhere above the target density.

The rejection-sampling algorithm is:

1. draw a proposal value x from $g(x)$;
2. draw $u \sim U(0, 1)$;
3. accept x if
$$u < \frac{p(x)}{Mg(x)};$$
otherwise reject the proposal and return to step 1.

The acceptance probability is largest where the envelope closely follows the target distribution. Efficient rejection sampling therefore requires a proposal distribution $g(x)$ that resembles $p(x)$ as closely as possible while remaining easy to draw from.

The correctness of the method can be understood geometrically. The accepted points are distributed uniformly beneath the curve $Mg(x)$. Retaining only those points that also lie beneath $p(x)$ leaves a set of points whose horizontal coordinates are distributed according to $p(x)$. Thus the accepted values have the target density $p(x)$.

The average acceptance rate is
$$\frac{1}{M}.$$
A large value of M therefore leads to many rejected proposals and unnecessary computational work. Much of the practical art of rejection sampling consists of constructing efficient envelopes.

Rejection sampling is especially useful when inverse-transform sampling is difficult but when it is easy to construct a proposal distribution that resembles the desired target distribution. More sophisticated Monte Carlo methods, including the Metropolis–Hastings algorithm, can be viewed as extensions of this basic idea.

Example: sampling from a parabolic density

Consider the probability density
$$p(x) = \frac{3}{4}(1 - x^2), \qquad -1 \le x \le 1. \tag{1.25}$$
It is normalized because
$$\int_{-1}^{1} p(x)\,\mathrm{d}x = \frac{3}{4}\int_{-1}^{1}(1 - x^2)\,\mathrm{d}x = 1. \tag{1.26}$$
We use the uniform proposal density
$$g(x) = \frac{1}{2}, \qquad -1 \le x \le 1. \tag{1.27}$$
The target density has its maximum at $x = 0$, where
$$p(0) = \frac{3}{4}. \tag{1.28}$$

Therefore, choosing $M = 3/2$ gives

$$Mg(x) = \frac{3}{2}\frac{1}{2} = \frac{3}{4} \geq p(x) \tag{1.29}$$

throughout the interval.

To generate one accepted value, draw

$$x \sim U(-1, 1), \qquad u \sim U(0, 1). \tag{1.30}$$

The proposal x is accepted if

$$u < \frac{p(x)}{Mg(x)} = 1 - x^2. \tag{1.31}$$

Otherwise, it is rejected and a new pair (x, u) is generated.

Equivalently, one may define

$$y = \frac{3}{4}u. \tag{1.32}$$

The pair (x, y) is then uniformly distributed in the rectangle

$$-1 \leq x \leq 1, \qquad 0 \leq y \leq \frac{3}{4}, \tag{1.33}$$

and the proposal is accepted when

$$y < p(x). \tag{1.34}$$

The accepted points are uniformly distributed beneath the curve $p(x)$, so their x-coordinates have the required probability density.

The mean acceptance rate is

$$\frac{1}{M} = \frac{2}{3}. \tag{1.35}$$

Thus, on average, two thirds of the proposed values are accepted.

1.3 Markov Chains

1.3.1 From independent draws to dependent states

The preceding methods generate independent draws: once the distribution and random-number generator are specified, one draw does not depend on the previous draw. Important problems in computational physics are not so easily treated. The state produced at one step may instead be obtained by modifying the state at the preceding step.

A particle moving between the two sides of a double-well potential is a simple example. If the particle is currently in the left well, it is likely to remain there at the next short time step; reaching the right well requires crossing a barrier. Consecutive states are therefore correlated. Nevertheless, under suitable conditions, long-time averages still converge to averages over a well-defined stationary distribution. This observation is the basis of Markov-chain Monte Carlo.

1.3.2 Discrete and continuous state spaces

A stochastic variable may be discrete, such as an Ising spin or an energy-level index, or continuous, such as a particle coordinate. Markov chains can be formulated for either type of state space.

For a discrete state space, transition probabilities are represented by a matrix. For a continuous state space, they are represented by a transition density or kernel. A continuous coordinate may also be discretized into bins of width Δx, with transitions between neighbouring bins. Decreasing Δx gives a finer approximation, at the cost of greater memory use and computational time.

1.3.3 The Markov property and transition probabilities

Let X_n denote the state after step n. A stochastic process is a first-order Markov chain if the probability distribution of the next state depends on the present state but not on the earlier history:

$$\Pr(X_{n+1} = j \mid X_n = i, X_{n-1}, \ldots, X_0) = \Pr(X_{n+1} = j \mid X_n = i). \tag{1.36}$$

For a finite set of states, define the transition matrix T by

$$T_{ij} = \Pr(X_{n+1} = j \mid X_n = i). \tag{1.37}$$

With this convention, each row sums to one:

$$T_{ij} \geq 0, \qquad \sum_j T_{ij} = 1. \tag{1.38}$$

If $p_i^{(n)} = \Pr(X_n = i)$ and $\boldsymbol{p}^{(n)}$ is written as a row vector, then

$$\boldsymbol{p}^{(n+1)} = \boldsymbol{p}^{(n)} T. \tag{1.39}$$

Thus the transition rule, together with the initial distribution, determines the evolution of the probability distribution.

1.3.4 Stationary distributions and convergence

A distribution $\boldsymbol{\pi}$ is stationary if one Markov step leaves it unchanged:

$$\boxed{\boldsymbol{\pi} = \boldsymbol{\pi} T}, \qquad \sum_i \pi_i = 1. \tag{1.40}$$

A stationary distribution is therefore a left eigenvector of T with eigenvalue one.

The existence of a stationary distribution does not by itself guarantee that the chain converges to it from every initial state. For a finite chain, a common sufficient set of conditions is:

- **Irreducibility:** every state can eventually be reached from every other state;
- **Aperiodicity:** the chain is not forced to return to a state only at multiples of a fixed integer greater than one.

For a finite Markov chain, irreducibility and aperiodicity imply that there exists a unique stationary distribution. Moreover, the distribution of the chain converges to this stationary distribution, regardless of the initial state. Such a chain is often called *ergodic*.

After an initial transient, or *burn-in*, the states of an ergodic chain can be used to estimate stationary averages. For an observable $A(i)$,

$$\langle A \rangle_\pi = \sum_i \pi_i A(i) \tag{1.41}$$

is estimated by the time average

$$\overline{A}_N = \frac{1}{N} \sum_{n=1}^{N} A(X_n). \tag{1.42}$$

Under suitable ergodicity conditions, $\overline{A}_N \to \langle A \rangle_\pi$ as $N \to \infty$. This is a Markov-chain version of the law of large numbers. Successive states are correlated, however, so their statistical efficiency is lower than that of the same number of independent observations.

1.3.5 Detailed balance

A transition matrix satisfies detailed balance with respect to $\boldsymbol{\pi}$ if

$$\boxed{\pi_i T_{ij} = \pi_j T_{ji}} \qquad \text{for every pair } i, j. \tag{1.43}$$

The two sides are the equilibrium probability fluxes from i to j and from j to i. Detailed balance therefore states that these pairwise fluxes cancel.

Summing over i gives

$$\sum_i \pi_i T_{ij} = \sum_i \pi_j T_{ji} \tag{1.44}$$

$$= \pi_j \sum_i T_{ji} \tag{1.45}$$

$$= \pi_j. \tag{1.46}$$

In the final step, $\sum_i T_{ji} = 1$ because i labels all possible destination states for a transition starting from state j. The term $\pi_i T_{ij}$ is the probability of initially being in state i and then making a transition from i to j. Summing over all possible initial states i, the left-hand side in the above expression,

$$\sum_i \pi_i T_{ij},$$

is therefore the total probability of being in state j after one Markov step. The right-hand side is π_j, the probability of being in state j before the step. Thus, if the distribution is initially $\boldsymbol{\pi}$, it remains $\boldsymbol{\pi}$ after one step. This is precisely the definition of stationarity. Detailed balance therefore implies stationarity.

Detailed balance is a sufficient condition for stationarity, not a necessary one. A stationary system may contain circulating probability currents and therefore violate detailed balance. Nevertheless, detailed balance is especially useful because it provides a direct way to design transition rules with a chosen target distribution. The Metropolis–Hastings algorithm, introduced later, exploits precisely this idea.

1.3.6 Example: a two-state chain

Consider two states, L and R, with transition matrix

$$T = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}, \qquad 0 \le \alpha, \beta \le 1. \tag{1.47}$$

Here, α is the probability of moving from L to R in one step, and β is the probability of moving from R to L. Solving $\boldsymbol{\pi} = \boldsymbol{\pi} T$ gives

$$\pi_L = \frac{\beta}{\alpha + \beta}, \qquad \pi_R = \frac{\alpha}{\alpha + \beta}, \tag{1.48}$$

provided $\alpha + \beta > 0$. The detailed-balance equation

$$\pi_L \alpha = \pi_R \beta \tag{1.49}$$

is satisfied. If $0 < \alpha, \beta < 1$, the chain is irreducible and aperiodic, so the long-run fractions of time spent in L and R converge to π_L and π_R.

This example captures the central idea: prescribed local transitions generate a global stationary distribution. In realistic simulations, the state can be a high-dimensional configuration rather than a single label, but the same principles apply.